

PYANSWER · ИТОГОВЫЙ КОМПЛЕКТ ДЛЯ ПРАКТИКИ

Модель авторизации и роли PyAnswer

Два обязательных материала в одном структурированном документе

Основано на commit [cc88219998c0ee34a7d7fafd744b5bf79e7865a5](#)
Alembic head [20260716_0006](#)
16 июля 2026 г.

1. Титульный блок

Проект: PyAnswer — локальная поисковая RAG-система по русскоязычной базе знаний Python.

Материалы: (1) модель авторизации; (2) роли и права доступа.

Срез: commit `cc88219998c0ee34a7d7fafd744b5bf79e7865a5`, Alembic `20260716_0006`.

2. Назначение документа

Документ фиксирует только фактически реализованную authentication/RBAC boundary, доказательства кода и тестов, route/API matrices и безопасные screenshots. Бизнес-логика и данные проекта не менялись.

3. Краткое описание системы PyAnswer

React обращается к FastAPI с `credentials: include`. PostgreSQL хранит users, roles, permissions, role_permissions, sessions и audit_events. Auth — server-side opaque session; authorization — permissions, загруженные по role.

Источник истины. Фактический backend-код → Alembic → автоматические тесты → frontend → документация. Проверены routes, services, dependencies/middleware, ORM и repositories, Pydantic schemas, migrations, React auth boundary, HTTP adapter, compose и только шаблон переменных окружения. Реальные секреты не читались и не включались.

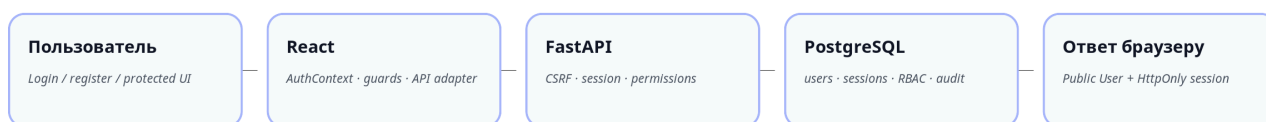
Часть 1. Модель авторизации

4. Модель авторизации

PyAnswer использует cookie-auth без Bearer/access token в browser storage. Identity хранится в PostgreSQL; браузер получает только opaque session reference в HttpOnly-cookie и отдельный CSRF token. Проверка выполняется заново на каждом запросе.

Общая схема авторизации PyAnswer

Trust boundary: browser хранит только opaque reference; решение принимает FastAPI

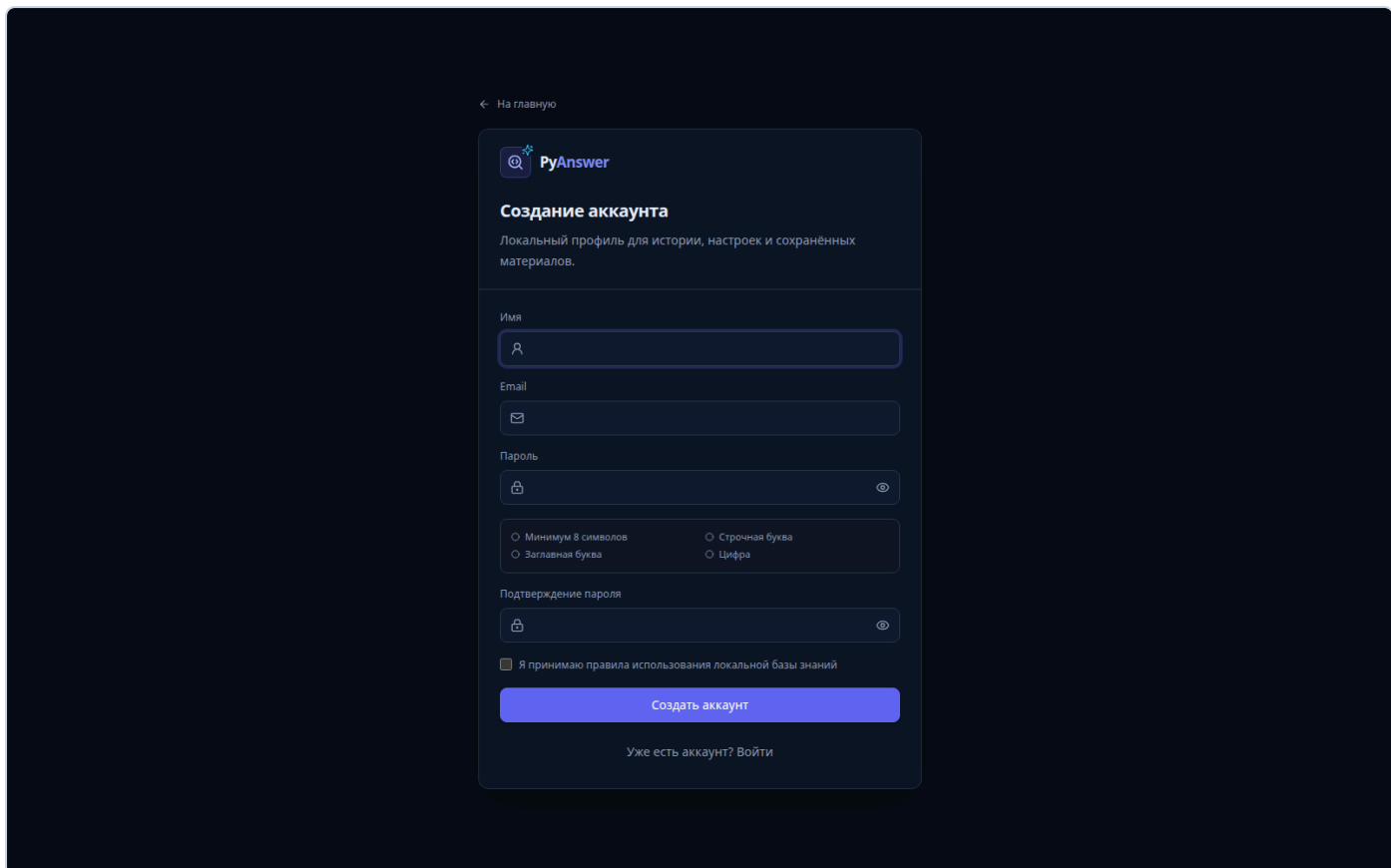


PyAnswer · commit cc88219998c0 · фактическая реализация

5. Регистрация пользователя

Поток: React form → `POST /api/auth/register` → Pydantic/backend validation → нормализация email (trim + casefold) → проверка уникальности → backend password policy → Argon2id → `password_hash` → роль USER → preferences → server session → cookies → публичный User.

- Роль в request schema отсутствует; extra-поле `role` отклоняется. Новый пользователь всегда USER.
- Backend повторно проверяет password policy; frontend validation — только UX.
- Raw password не сохраняется. UserOut и ADMIN responses не содержат hash.
- REGISTER audit содержит безопасные identity-поля, но не credential.



6. Хранение паролей

Используется `argon2-cffi PasswordHasher`; фактический тест проверяет префикс Argon2id, verify и отказ для неверного пароля. Соль и параметры кодируются в стандартной строке Argon2. Raw password живёт только в памяти обработки запроса; request body не журналируется. ADMIN не получает `password_hash`.

Неправильно (синтетический пример):

```
user: admin · password: my_password_123
```

Правильно (не значение из БД):

```
user: admin · password_hash: $argon2id$...
```

Audit sanitizer рекурсивно удаляет ключи, содержащие password, hash, salt, digest, session, csrf, cookie, authorization, secret, api_key/apikey и token. Logging filter также редактирует credential keys.

7. Вход и создание серверной сессии

После успешного Argon2 verify сервер генерирует два независимых криптографически случайных opaque token: session и CSRF. Клиент получает session token только в HttpOnly-cookie. В таблице `sessions` остаётся SHA-256 `token_hash`, а также hash CSRF, user id, snapshot `account_version`, remember flag, expiry/revocation timestamps и служебная metadata.

При запросе: cookie → SHA-256 → поиск session → проверки `revoked_at`, `expires_at`, ACTIVE status и равенства account versions. Только затем доступен actor.

Регистрация, вход и серверная сессия

Raw password и raw session token не сохраняются



PyAnswer · commit cc8219998c0 · фактическая реализация

8. Cookie и CSRF

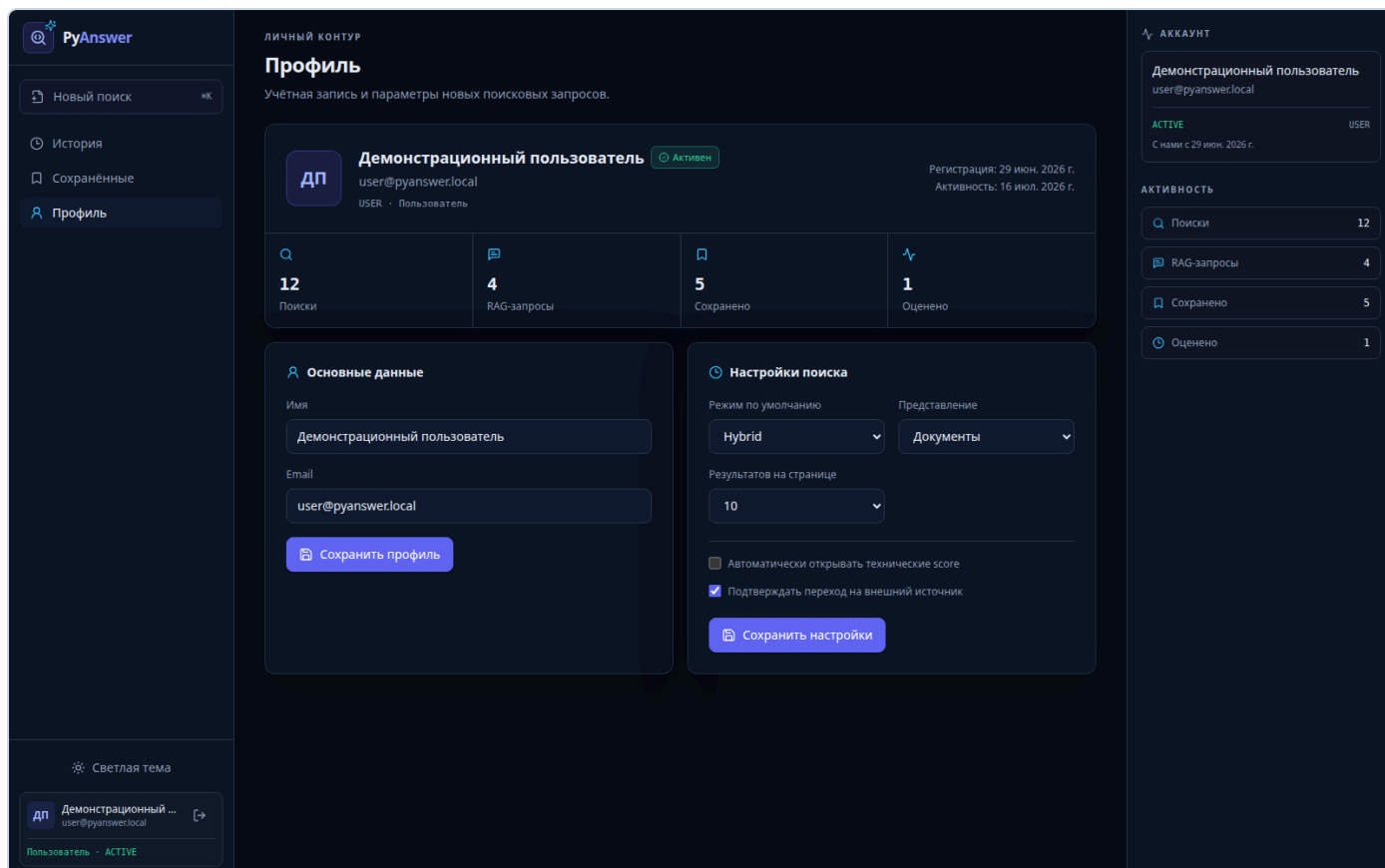
Session cookie: **HttpOnly**, Secure по конфигурации (обязателен в production), SameSite из allowlist, Path= / ; Max-Age задаётся для rememberMe. Обычная сессия имеет session-cookie без постоянного Max-Age, но всегда имеет серверный expiry. Конкретные cookie values в комплект не включены.

CSRF требуется из-за автоматической отправки cookie браузером. Middleware защищает POST/PUT/PATCH/DELETE под `/api/`: readable CSRF cookie должна совпасть с `X-CSRF-Token`; authenticated dependency дополнительно сравнивает hash с текущей server session. GET не меняет состояние. Login/register создают новую session/CSRF пару; adapter очищает старый in-memory token. При CSRF_INVALID adapter получает новый token и выполняет не более одного retry.

Цепочка: Frontend → GET CSRF → mutation + session cookie + X-CSRF-Token → middleware double-submit → session lookup → session-bound CSRF hash → действие.

9. Проверка текущей сессии

AuthContext вызывает `GET /api/auth/me`. Пока query pending, status=initializing и защищённый контент не рендерится. Валидная session возвращает публичный User с role/permissions; отсутствие session возвращает `null`, а защищённый endpoint — 401. На 401 HTTP adapter уведомляет AuthContext; user/role-sensitive TanStack Query roots (user, history, saved, feedback, search, document, editor, admin) удаляются.



10. Выход и отзыв сессии

POST `/api/auth/logout` требует действующую session и CSRF. Server выставляет `revoked_at`, пишет LOGOUT audit и удаляет session/CSRF cookies. Отзыванная запись остаётся доказательством состояния и больше не аутентифицирует.

11. Блокировка пользователя

ADMIN → USERS_MANAGE → reason validation → self-block check → last-active-ADMIN check → status=BLOCKED → account_version++ → revoke sessions → BLOCK_USER audit. BLOCKED login фактически возвращает **403 FORBIDDEN**; старая session становится anonymous и protected API возвращает 401. Unblock сохраняет пользовательские данные, меняет status на ACTIVE, снова повышает version и пишет audit; следующая авторизация создаёт новую session.

12. Изменение роли и инвалидирование сессий

Role приходит только в защищённый admin request, но actor берётся из server session. Target читается с row lock. При изменении role повышается account_version, все target sessions отзываются, до/после сохраняются в audit. Поэтому старый browser context не продолжает работу с прежними правами.

13. Защита последнего администратора

Self-role-change (включая self-demotion) и self-block дают 409. Для операций над ADMIN реально используется PostgreSQL transaction-scoped advisory lock, target row lock и подсчёт ACTIVE ADMIN. Последнего активного ADMIN нельзя заблокировать или понизить. FastAPI DB dependency commit/rollback делает изменение, revoke и audit одной транзакционной единицей.

Изменение роли и блокировка

Операция, revoke sessions и audit входят в одну transaction boundary



PyAnswer · commit cc8219998c0 · фактическая реализация

14. Журнал аудита

AuditEvent хранит actor id/name/role, action, entity type/id/label, outcome, summary, before/after/metadata, request ID и created_at; IP может храниться сервером, но в этот документ не включён. Реальные actions: LOGIN, LOGOUT, REGISTER, CHANGE_USER_ROLE, BLOCK_USER, UNBLOCK_USER, UPDATE_DOCUMENT_METADATA, HIDE_DOCUMENT, RESTORE_DOCUMENT, REINDEX_DOCUMENT, START_SOURCE_SYNC, CANCEL_JOB, UPDATE_SYSTEM_SETTINGS и другие enum values. Audit API read-only и требует AUDIT_VIEW.

PyAnswer

Демонстрационный администратор

Обзор администратора

Пользователи

Источники

Все задания

Поисковый индекс

Локальный RAG

Аудит

Система

Раздел редактора

Вернуться к поиску

ПАНЕЛЬ УПРАВЛЕНИЯ

Администратор / Аудит

IMMUTABLE AUDIT LOG

Журнал аудита

События создаются внутри business/mock repository вместе с операцией. Редактирование и удаление журнала не предусмотрено.

Событие, сущность, requestId

ALL

ALL

ALL

Время	Актор	Роль	Описание	Сущность	Результат	Ссылка	Request ID
16.07.2026, 23:01:05	Демонстрационный администратор	AUTH	admin@pyanswer.local		Успешно		5ba22bd7-563b-4d95-b820f62683e1
16.07.2026, 23:01:05	Демонстрационный администратор	AUTH	admin@pyanswer.local		Успешно		1090c-108-b8c2796b-e980-4312-81e5-f754e0f66462
16.07.2026, 23:01:04	Демонстрационный редактор	EDITOR	HIDE_DOCUMENT	DOCUMENT	Удаление с лк прилож...	Успешно	646f09a-8052-4a2b-b8f1-3a33a852a28
16.07.2026, 23:01:03	Демонстрационный редактор	EDITOR	UPDATE_DOCUMENT_METADATA	DOCUMENT	Удаление с лк прилож...	Успешно	87780582-2523-4741-a6c9-83891a531a60
16.07.2026, 23:00:53	Демонстрационный редактор	EDITOR	RESTORE_DOCUMENT	DOCUMENT	Удаление с лк прилож...	Успешно	734de320-359c-4c95-a07b-f99828577641
16.07.2026, 23:00:38	Демонстрационный редактор	EDITOR	LOGIN	AUTH	Удаление с лк прилож...	Успешно	e1c26516-6515-4541-a845-d60797251875
16.07.2026, 23:00:33	Демонстрационный редактор	EDITOR	LOGIN	AUTH	Удаление с лк прилож...	Успешно	864e31c-8a78-4778-b769-853885af0e4
16.07.2026, 23:00:31	Демонстрационный редактор	EDITOR	LOGIN	AUTH	Удаление с лк прилож...	Успешно	2853879-ac12-417e-a9cc-4c212878a34
16.07.2026, 23:00:21	Демонстрационный редактор	EDITOR	LOGIN	AUTH	Удаление с лк прилож...	Успешно	a78a07ba-bc41-4515-856d-c5a1278a745
16.07.2026, 22:59:17	Демонстрационный редактор	EDITOR	REINDEX_DOCUMENT	DOCUMENT	Удаление с лк прилож...	Успешно	1090c-108-949a986a-79e7-4465-ba07-15d39081e095

Часть 2. Роли и права доступа

15. Роли системы

USER

Обычный пользователь: BM25/Vector/Hybrid search, RAG, публичные документы, собственные profile/preferences, history, saved и feedback. Регистрация выдаёт только эту роль.

EDITOR

Наследует USER. Дополнительно: editor dashboard, все управляемые документы (включая скрытые/ошибочные), metadata/tags/internal note, hide/restore, document reindex и доступные editor jobs. Не управляет users/roles, sources, admin jobs, audit или system settings.

ADMIN

Наследует EDITOR. Дополнительно: admin dashboard, users/roles/blocking, sources/sync, jobs/retry/cancel/FULL_REINDEX, audit, system status/settings, index versions/blue-green operations и RAG diagnostics.

16. Permission matrix

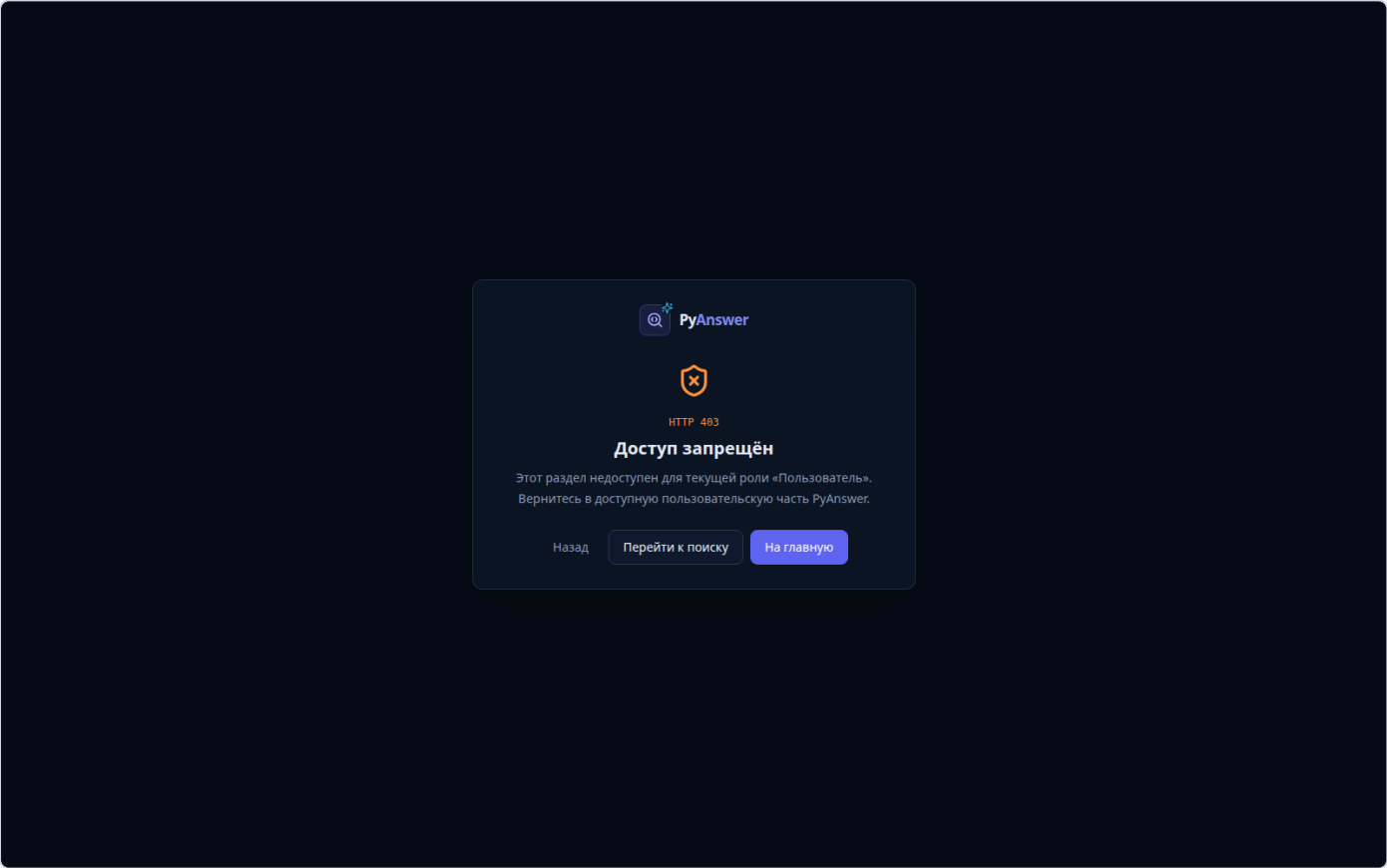
Матрица совпадает в backend `core/permissions.py`, enum, bootstrap и frontend `features/auth/permissions.ts`. В live PostgreSQL: 3 роли, 20 permissions.

Permission	USER	EDITOR	ADMIN	Назначение
SEARCH_USE	Да	Да	Да	Поиск BM25, Vector и Hybrid
RAG_USE	Да	Да	Да	Локальный RAG
PROFILE_MANAGE	Да	Да	Да	Свой профиль и preferences
HISTORY_MANAGE	Да	Да	Да	Своя история
SAVED_MANAGE	Да	Да	Да	Свои сохранённые документы
EDITOR_ACCESS	Нет	Да	Да	Frontend-вход в контур EDITOR
MANAGED_DOCUMENTS_VIEW	Нет	Да	Да	Управляемые документы и dashboard
DOCUMENT_METADATA_EDIT	Нет	Да	Да	Редакторские метаданные
DOCUMENT_STATUS_CHANGE	Нет	Да	Да	Hide/restore
DOCUMENT_REINDEX	Нет	Да	Да	Переиндексация документа
EDITOR_JOBS_VIEW	Нет	Да	Да	Доступные редактору jobs
ADMIN_ACCESS	Нет	Нет	Да	Frontend/API dashboard ADMIN
USERS_MANAGE	Нет	Нет	Да	Пользователи, роли, блокировки
SOURCES_MANAGE	Нет	Нет	Да	Источники и ingestion

Permission	USER	EDITOR	ADMIN	Назначение
ADMIN_JOBS_MANAGE	Нет	Нет	Да	Все jobs и FULL_REINDEX
AUDIT_VIEW	Нет	Нет	Да	Read-only audit
SYSTEM_VIEW	Нет	Нет	Да	System status, settings view, RAG diagnostics
SYSTEM_SETTINGS_MANAGE	Нет	Нет	Да	Изменение system settings
SEARCH_INDEX_VIEW	Нет	Нет	Да	Версии и статистика индексов
SEARCH_INDEX_MANAGE	Нет	Нет	Да	Blue-green index operations

17. Проверка прав на frontend

ProtectedRoute проверяет завершённую AuthContext session; PermissionRoute требует все указанные permissions. USER не видит editor/admin navigation; EDITOR видит editor, но не admin; ADMIN видит оба контура. Гость получает login redirect с безопасным internal returnUrl; authenticated actor без permission — /403. Это UX boundary, а не доверенная защита.

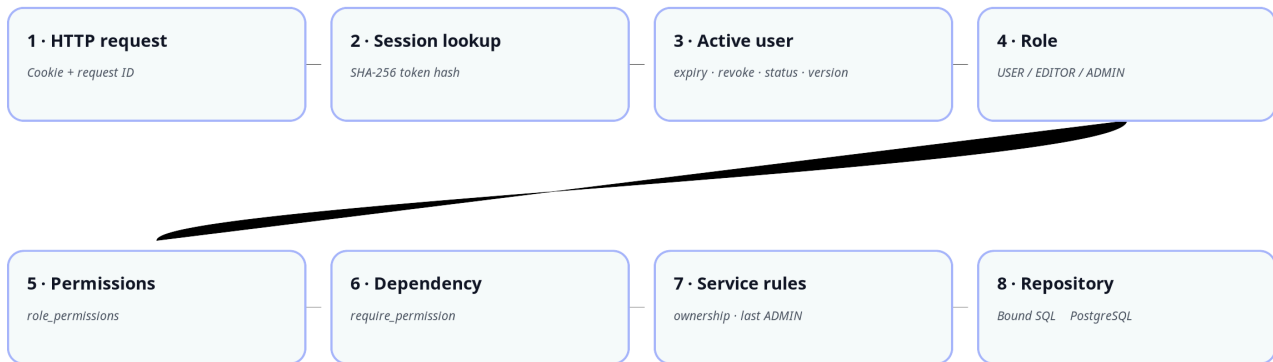


18. Проверка прав на backend

Backend игнорирует role/query/header injection: actor формируется только по session. Цепочка: HTTP → cookie → token hash lookup → active user/status/version → role → role_permissions → require_permission → service rules → repository → PostgreSQL. Editor/admin endpoints используют named permission dependencies; service layer отдельно применяет ownership, self-action и last-admin rules. Прямой запрещённый вызов возвращает 403.

Проверка permissions на backend

Роль из URL, header или payload не считается доверенной



PyAnswer · commit cc8219998c0 · фактическая реализация

Пять базовых USER permission присутствуют в матрице и ответе User. Profile/history/saved/feedback endpoints фактически защищены CurrentUser и ownership, а не отдельным `require_permission`; search/RAG проверяют SEARCH_USE/RAG_USE вручную. Это зафиксированная деталь реализации, не расширение модели.

19. Матрица доступа к страницам

Всего 15 frontend routes под ProtectedRoute/PermissionRoute: 3 USER + 4 EDITOR + 8 ADMIN.

Маршрут	Гость	USER	EDITOR	ADMIN
/	Доступен	Доступен	Доступен	Доступен
/search	Доступен	Доступен	Доступен	Доступен
/documents/:documentId	Доступен	Доступен	Доступен	Доступен
/login	Доступен	Доступен	Доступен	Доступен
/register	Доступен	Доступен	Доступен	Доступен
/profile	Redirect /login	Доступен	Доступен	Доступен
/history	Redirect /login	Доступен	Доступен	Доступен
/saved	Redirect /login	Доступен	Доступен	Доступен
/editor	Redirect /login	403	Доступен	Доступен
/editor/documents	Redirect /login	403	Доступен	Доступен
/editor/documents/:documentId	Redirect /login	403	Доступен	Доступен
/editor/jobs	Redirect /login	403	Доступен	Доступен
/admin	Redirect /login	403	403	Доступен
/admin/users	Redirect /login	403	403	Доступен

Маршрут	Гость	USER	EDITOR	ADMIN
/admin/sources	Redirect /login	403	403	Доступен
/admin/jobs	Redirect /login	403	403	Доступен
/admin/audit	Redirect /login	403	403	Доступен
/admin/system	Redirect /login	403	403	Доступен
/admin/indexes	Redirect /login	403	403	Доступен
/admin/rag	Redirect /login	403	403	Доступен

20. Матрица доступа к API

Строго session-protected endpoints: **63** (13 authenticated-user, 13 editor, 37 admin). Условно публичные search/ask зависят от server policy и в 63 не включены.

Метод	Endpoint	Гость	USER	EDITOR	ADMIN	Permission / правило
GET	/api/auth/csrf	Да	Да	Да	Да	Публично; CSRF token
POST	/api/auth/register	Да + CSRF	Да + CSRF	Да + CSRF	Да + CSRF	Публично; всегда создаёт USER
POST	/api/auth/login	Да + CSRF	Да + CSRF	Да + CSRF	Да + CSRF	Публично
POST	/api/auth/logout	401	Да	Да	Да	CurrentUser + CSRF
GET	/api/auth/me	null	Да	Да	Да	OptionalUser
GET	/api/users/me	401	Да	Да	Да	CurrentUser; ownership
PATCH	/api/users/me	401	Да	Да	Да	CurrentUser; ownership
GET	/api/users/me/stats	401	Да	Да	Да	CurrentUser; ownership
GET	/api/history	401	Да	Да	Да	CurrentUser; ownership
DELETE	/api/history	401	Да	Да	Да	CurrentUser; ownership
DELETE	/api/history/{id}	401	Да	Да	Да	CurrentUser; ownership
GET	/api/saved	401	Да	Да	Да	CurrentUser; ownership
POST	/api/saved/{documentId}	401	Да	Да	Да	CurrentUser; ownership
DELETE	/api/saved/{documentId}	401	Да	Да	Да	CurrentUser; ownership
POST	/api/feedback	401	Да	Да	Да	CurrentUser; ownership
GET	/api/feedback/by-response/{id}	401	Да	Да	Да	CurrentUser; ownership
DELETE	/api/feedback/{id}	401	Да	Да	Да	CurrentUser; ownership
GET	/api/documents/{id}	Да	Да	Да	Да	Публичный ACTIVE document
GET	/api/search	По policy	Да	Да	Да	SEARCH_USE для actor
POST	/api/ask	По policy + CSRF	Да	Да	Да	RAG_USE для actor + CSRF
POST	/api/ask/stream	По policy + CSRF	Да	Да	Да	RAG_USE для actor + CSRF

Метод	Endpoint	Гость	USER	EDITOR	ADMIN	Permission / правило
GET	/api/editor/dashboard	401	403	Да	Да	MANAGED_DOCUMENTS_VIEW
GET	/api/editor/documents	401	403	Да	Да	MANAGED_DOCUMENTS_VIEW
POST	/api/editor/documents/bulk	401	403	Да	Да	DOCUMENT_STATUS_CHANGE + DOCUMENT_REINDEX
GET	/api/editor/documents/{id}/chunks	401	403	Да	Да	MANAGED_DOCUMENTS_VIEW
GET	/api/editor/documents/{id}/revisions	401	403	Да	Да	MANAGED_DOCUMENTS_VIEW
GET	/api/editor/documents/{id}/failures	401	403	Да	Да	MANAGED_DOCUMENTS_VIEW
GET	/api/editor/documents/{id}	401	403	Да	Да	MANAGED_DOCUMENTS_VIEW
PATCH	/api/editor/documents/{id}/metadata	401	403	Да	Да	DOCUMENT_METADATA_EDIT
POST	/api/editor/documents/{id}/hide	401	403	Да	Да	DOCUMENT_STATUS_CHANGE
POST	/api/editor/documents/{id}/restore	401	403	Да	Да	DOCUMENT_STATUS_CHANGE
POST	/api/editor/documents/{id}/reindex	401	403	Да	Да	DOCUMENT_REINDEX
GET	/api/editor/jobs/{id}/events	401	403	Да	Да	EDITOR_JOBS_VIEW
GET	/api/editor/jobs	401	403	Да	Да	EDITOR_JOBS_VIEW
GET	/api/admin/dashboard	401	403	403	Да	ADMIN_ACCESS
GET	/api/admin/users	401	403	403	Да	USERS_MANAGE
GET	/api/admin/users/{id}	401	403	403	Да	USERS_MANAGE
PATCH	/api/admin/users/{id}/role	401	403	403	Да	USERS_MANAGE
POST	/api/admin/users/{id}/block	401	403	403	Да	USERS_MANAGE
POST	/api/admin/users/{id}/unblock	401	403	403	Да	USERS_MANAGE
GET	/api/admin/sources	401	403	403	Да	SOURCES_MANAGE
GET	/api/admin/sources/{id}/sync-state	401	403	403	Да	SOURCES_MANAGE
GET	/api/admin/sources/{id}	401	403	403	Да	SOURCES_MANAGE
PATCH	/api/admin/sources/{id}	401	403	403	Да	SOURCES_MANAGE
POST	/api/admin/sources/{id}/test	401	403	403	Да	SOURCES_MANAGE
POST	/api/admin/sources/{id}/sync	401	403	403	Да	SOURCES_MANAGE
POST	/api/admin/sources/{id}/stop	401	403	403	Да	SOURCES_MANAGE
GET	/api/admin/ingestion/stats	401	403	403	Да	SOURCES_MANAGE
GET	/api/admin/ingestion/failures	401	403	403	Да	SOURCES_MANAGE
GET	/api/admin/ingestion/failures/{id}	401	403	403	Да	SOURCES_MANAGE
GET	/api/admin/jobs	401	403	403	Да	ADMIN_JOBS_MANAGE
POST	/api/admin/jobs/full-reindex	401	403	403	Да	ADMIN_JOBS_MANAGE
GET	/api/admin/jobs/{id}/events	401	403	403	Да	ADMIN_JOBS_MANAGE
GET	/api/admin/jobs/{id}	401	403	403	Да	ADMIN_JOBS_MANAGE

Метод	Endpoint	Гость	USER	EDITOR	ADMIN	Permission / правило
POST	/api/admin/jobs/{id}/retry	401	403	403	Да	ADMIN_JOBS_MANAGE
POST	/api/admin/jobs/{id}/cancel	401	403	403	Да	ADMIN_JOBS_MANAGE
GET	/api/admin/audit	401	403	403	Да	AUDIT_VIEW
GET	/api/admin/audit/{id}	401	403	403	Да	AUDIT_VIEW
GET	/api/admin/system	401	403	403	Да	SYSTEM_VIEW
POST	/api/admin/system/health-check	401	403	403	Да	SYSTEM_VIEW
GET	/api/admin/system/settings	401	403	403	Да	SYSTEM_VIEW
PATCH	/api/admin/system/settings	401	403	403	Да	SYSTEM_SETTINGS_MANAGE
GET	/api/admin/indexes	401	403	403	Да	SEARCH_INDEX_VIEW
GET	/api/admin/indexes/active	401	403	403	Да	SEARCH_INDEX_VIEW
GET	/api/admin/indexes/stats	401	403	403	Да	SEARCH_INDEX_VIEW
POST	/api/admin/indexes/full-reindex	401	403	403	Да	SEARCH_INDEX_MANAGE
POST	/api/admin/indexes/cleanup	401	403	403	Да	SEARCH_INDEX_MANAGE
GET	/api/admin/indexes/{id}	401	403	403	Да	SEARCH_INDEX_VIEW
POST	/api/admin/indexes/{id}/validate	401	403	403	Да	SEARCH_INDEX_MANAGE
GET	/api/admin/rag	401	403	403	Да	SYSTEM_VIEW
POST	/api/admin/rag/test	401	403	403	Да	SYSTEM_VIEW

21. Принцип минимальных привилегий

- USER не изменяет managed documents и не открывает management API.
- EDITOR ограничен documents и editor-visible jobs; users, sources, audit и system недоступны.
- ADMIN имеет системные permissions, но не может изменить собственную роль или заблокировать себя.
- Система не может остаться без ACTIVE ADMIN; смена role и block инвалидируют sessions.
- Роль назначается только через USERS_MANAGE endpoint; register не принимает role.
- Audit read-only; UI hiding не заменяет server check.

Возможность	USER	EDITOR	ADMIN
Поиск BM25 / Vector / Hybrid	Да	Да	Да
RAG	Да	Да	Да
Профиль, история, saved, feedback	Да	Да	Да
Публичные документы	Да	Да	Да
Editor dashboard	Нет	Да	Да
Управление документами и metadata	Нет	Ограниченно	Да
Hide / restore	Нет	Да	Да
Document reindex	Нет	Да	Да
Editor jobs	Нет	Только просмотр	Да

Возможность	USER	EDITOR	ADMIN
Admin dashboard	Нет	Нет	Да
Управление пользователями	Нет	Нет	Да
Изменение ролей	Нет	Нет	Ограниченно
Блокировка / разблокировка	Нет	Нет	Ограниченно
Источники и source sync	Нет	Нет	Да
Job retry / cancel	Нет	Нет	Да
FULL_REINDEX	Нет	Нет	Да
Audit	Нет	Нет	Только просмотр
System status	Нет	Нет	Только просмотр
System settings	Нет	Нет	Да
Управление ролями	Нет	Нет	Ограниченно

22. Обработка ошибок 401 и 403

401 Unauthorized: session отсутствует, истекла, отозвана или стала invalid из-за account_version/status. **403 Forbidden:** actor аутентифицирован, но нет permission; также фактический login BLOCKED-user возвращает 403 и CSRF failure возвращает 403 с CSRF_INVALID.

Backend ApiError envelope → HTTP adapter ApiError → 401 subscriber очищает auth/cache и guard ведёт на login; 403 сохраняется typed error и PermissionRoute ведёт на страницу 403.

23. Результаты тестирования

Профильный запуск на disposable test DB: **46 passed**. Frontend Vitest security/guards: **23 passed**. Полный suite не запускался в рамках этой проверки.

Требование	Тест	Файл	Результат
Argon2id / raw password	test_argon2id_hash_verify_and_reject_raw_password	backend/tests/unit/test_security.py	PASS
Register создаёт USER; duplicate; HttpOnly	test_registration_creates_only_user_and_casefolds_email	backend/tests/integration/test_auth_api.py	PASS
Role injection и слабый пароль	test_registration_rejects_role_and_weak_password	backend/tests/integration/test_auth_api.py	PASS
Login, hash token, /me, нет credentials	test_login_cookie_stores_only_hash_and_neutral_error	backend/tests/integration/test_auth_api.py	PASS
CSRF	test_csrf_is_required_and_validated	backend/tests/integration/test_auth_api.py	PASS
Expired / revoked / account_version	test_invalid_session_cannot_access_protected_api (3 cases)	backend/tests/integration/test_auth_api.py	PASS
BLOCKED login	test_blocked_user_cannot_login	backend/tests/integration/test_auth_api.py	PASS
Logout revoke	test_logout_requires_csrf_and_revokes_session	backend/tests/integration/test_auth_api.py	PASS
CORS / headers / request ID	test_unknown_cors_origin_is_not_allowed; test_request_id_security_headers_and_error_envelope	backend/tests/integration/test_auth_api.py	PASS

Требование	Тест	Файл	Результат
Rate limiter	<code>test_rate_limiter_returns_429_after_limit</code>	backend/tests/integration/test_auth_api.py	PASS
USER запрещён EDITOR/ADMIN	<code>test_user_cannot_call_editor_or_admin_even_with_role_injection</code>	backend/tests/integration/test_rbac_editor_api.py	PASS
EDITOR запрещён ADMIN	<code>test_editor_can_use_editor_but_not_admin</code>	backend/tests/integration/test_rbac_editor_api.py	PASS
ADMIN наследует EDITOR	<code>test_admin_inherits_editor_and_admin_permissions</code>	backend/tests/integration/test_rbac_editor_api.py	PASS
Self-block / self-role-change	<code>test_admin_cannot_change_role_or_block_self</code>	backend/tests/integration/test_admin_api.py	PASS
Role change: revoke + audit	<code>test_role_change_revokes_target_sessions_and_creates_audit</code>	backend/tests/integration/test_admin_api.py	PASS
Block/unblock/account_version/audit	<code>test_block_revokes_session_and_unblock_allows_login</code>	backend/tests/integration/test_admin_api.py	PASS
Last ADMIN	<code>test_last_active_admin_rule_is_locked_and_enforced</code>	backend/tests/integration/test_admin_api.py	PASS
Audit read-only + sanitized	<code>test_audit_is_admin_read_only_and_sanitized</code>	backend/tests/integration/test_admin_api.py	PASS
Recursive audit sanitizer	<code>test_audit_sanitizer_removes_credentials_recursively</code>	backend/tests/unit/test_security.py	PASS
Frontend 401/CSRF adapter	уведомляет auth store при 401; повторяет запрос после CSRF refresh только один раз	frontend/src/api/httpApi.test.ts	PASS
Frontend guards/navigation	<code>PermissionRoute</code> (9 cases); <code>ProtectedRoute</code> (1 case)	frontend/src/features/auth/*.test.tsx	PASS
Явный тест Max-Age rememberMe	Автоматический тест не найден	—	NOT CHECKED
Явное сравнение CSRF до/после login	Автоматический тест не найден	—	NOT CHECKED

24. Ограничения и дальнейшее развитие

- Auth rate limiter process-local и in-memory: для нескольких backend replicas нужен общий store.
- MFA, reset password, session inventory/revoke-all UI не реализованы.
- Нет отдельного автоматического assertion для rememberMe Max-Age и сравнения CSRF value до/после login; механизмы проверены чтением исполняемого кода.
- Base USER endpoints используют CurrentUser/ownership вместо uniform named permission dependency.
- Frontend README и отдельные UI descriptions сохраняют исторические mock/server-like формулировки; HTTP adapter и FastAPI уже реализованы.
- Frontend README admin permission list не содержит SEARCH_INDEX_VIEW/MANAGE, хотя actual backend/frontend code и bootstrap содержат их; фактическое число — 20.

25. Заключение

Критические условия выполнены: пароль только Argon2id hash; server-side session и HttpOnly cookie; CSRF; backend RBAC; три роли; last-admin guard; session invalidation; sanitized audit. Итог проверки: **PASS**, при документированных non-blocking ограничениях process-local rate limiter и

устаревшем тексте frontend documentation.